



The
University of
Faisalabad

Instructor:

Mam Eman Basharat

Submitted By:

Abdul Bari

Registration Number:

2024-BS-CS-289

Department:

Computer Science

Course:

Database Systems

Course Code:

CS-213 Lab

Table of Contents

LAB 1: Introduction to SQL	3
LAB 2: Creating Tables	6
LAB 3: Inserting and Retrieving Data	9
LAB 4: Data Constraints in SQL	13
LAB 5: Data Retrieval Using ‘WHERE’	20
LAB 6: Sorting and Filtering Data in SQL	26
LAB 7: Modifying Data and Aggregate Functions.....	31
LAB 8: Keys in DBMS	36
LAB 9: Filtering Data using ‘WHERE’ and ‘LIKE’	41
LAB 10: Advanced Data Filtering I.....	47
LAB 11: Advanced Data Filtering II	52
LAB 12: Normalization	56
LAB 13: Inner and Cross Joins	61
LAB 14: Outer Join and It’s Types.....	63

LAB 1: INTRODUCTION TO SQL

SQL (Structured Query Language) is the standard language used to interact with databases. It allows you to store, retrieve, update, and delete data in a **relational database**.

SQL is widely used in fields like data analysis, web development, backend systems, and business intelligence.

What is a Database?

A **database** is an organized collection of data. In SQL, data is stored in **tables**, which look like spreadsheets:

id	name	age
1	Abdul Bari	25
2	Ali	30

- **Rows** = records
- **Columns** = fields (attributes)

What Can SQL Do?

With SQL, you can:

- Query data (retrieve information)
- Insert new data
- Update existing data
- Delete data
- Create and modify tables and databases

Basic SQL Commands

1. SELECT (Retrieve Data)

```
SELECT name, age FROM users;
```

Gets specific columns from a table.

2. INSERT (Add Data)

```
INSERT INTO users (name, age)
VALUES ('Abdul Bari', 28);
```

3. UPDATE (Modify Data)

```
UPDATE users
SET age = 29
WHERE name = 'Abdul Bari';
```

4. DELETE (Remove Data)

```
DELETE FROM users
WHERE name = 'Abdul Bari';
```

5. CREATE TABLE

```
CREATE TABLE users (
    id INT,
    name VARCHAR(100),
    age INT
);
```

Key Concepts

Primary Key:

A unique identifier for each row (e.g., id).

WHERE Clause:

Filters data:

```
SELECT * FROM users WHERE age > 25;
```

ORDER BY:

Sorts results:

```
SELECT * FROM users ORDER BY age DESC;
```

JOIN:

Combines data from multiple tables:

```
SELECT users.name, orders.product  
FROM users  
JOIN orders ON users.id = orders.user_id;
```

Types of SQL Databases

Some popular systems include:

- MySQL
- PostgreSQL
- SQLite
- Microsoft SQL Server
- Oracle Database

Why Learn SQL?

- Essential for working with data
- Used in almost every tech company
- Works with tools like Python, Excel, and BI platforms
- High demand skill in data-related jobs

LAB 2: CREATING TABLES

Creating Tables in MySQL

In MySQL, tables are created using the `CREATE TABLE` statement.

Basic Syntax

```
CREATE TABLE table_name (  
    column1 datatype constraints,  
    column2 datatype constraints,  
    column3 datatype constraints  
);
```

Example: Create a Simple Table

```
CREATE TABLE students (  
    id INT PRIMARY KEY,  
    name VARCHAR(100),  
    age INT,  
    city VARCHAR(50)  
);
```

Explanation

Column	Description
id	Integer value, unique for each student
PRIMARY KEY	Ensures each row is unique
VARCHAR(100)	Text up to 100 characters
age	Integer number
city	Stores city name

Common Data Types

Data Type	Purpose
INT	Whole numbers
VARCHAR(n)	Variable-length text
TEXT	Large text

DATE	Dates
FLOAT	Decimal numbers
BOOLEAN	True/False values

Using AUTO_INCREMENT

Automatically generates IDs:

```
CREATE TABLE employees (  
    emp_id INT AUTO_INCREMENT PRIMARY KEY,  
    emp_name VARCHAR(100),  
    salary FLOAT  
);
```

When inserting data, you don't need to provide emp_id.

Adding Constraints NOT NULL

Prevents empty values:

```
CREATE TABLE users (  
    id INT PRIMARY KEY,  
    username VARCHAR(50) NOT NULL  
);
```

UNIQUE

Ensures no duplicate values:

```
email VARCHAR(100) UNIQUE
```

DEFAULT

Sets a default value:

```
status VARCHAR(20) DEFAULT 'active'
```

Complete Example

```
CREATE TABLE products (  
    product_id INT AUTO_INCREMENT PRIMARY KEY,  
    product_name VARCHAR(100) NOT NULL,  
    price FLOAT NOT NULL,  
    stock INT DEFAULT 0,  
    created_at DATE  
);
```

Example Workflow

```
CREATE DATABASE school;  
  
USE school;  
  
> CREATE TABLE students (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    name VARCHAR(100),  
    age INT  
);
```


LAB 3: INSERTING AND RETRIEVING DATA

Inserting and Retrieving Data in MySQL

Using MySQL, you can add data with INSERT and retrieve data with SELECT.

1. Inserting Data (INSERT INTO)

Basic Syntax

```
INSERT INTO table_name (column1, column2, column3)
VALUES (value1, value2, value3);
```

Example Table

```
CREATE TABLE students (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100),
    age INT,
    city VARCHAR(50)
);
```

Insert One Row

```
INSERT INTO students (name, age, city)
VALUES ('Ali', 20, 'Faisalabad');
```

Since id is AUTO_INCREMENT, MySQL generates it automatically.

Insert Multiple Rows

```
INSERT INTO students (name, age, city)
VALUES
('Sara', 22, 'Lahore'),
('Ahmed', 21, 'Karachi'),
('Ayesha', 19, 'Islamabad');
```

2. Retrieving Data (SELECT)

Retrieve All Data

```
SELECT * FROM students;
```

* means all columns.

	id	name	age	city
▶	1	Ali	20	Faisalabad
	2	Sara	22	Lahore
	3	Ahmed	21	Karachi
	4	Ayesha	19	Islamabad
*	NULL	NULL	NULL	NULL

Retrieve Specific Columns

```
SELECT name, city FROM students;
```

Using WHERE (Filtering Data) Students older than 20

```
SELECT * FROM students
WHERE age > 20;
```

	id	name	age	city
▶	2	Sara	22	Lahore
	3	Ahmed	21	Karachi
*	NULL	NULL	NULL	NULL

Find a specific student

```
SELECT * FROM students
WHERE name = 'Ali';
```

	id	name	age	city
▶	1	Ali	20	Faisalabad
*	NULL	NULL	NULL	NULL

Sorting Data (ORDER BY) Ascending Order

```
SELECT * FROM students
ORDER BY age ASC;
```

	id	name	age	city
▶	4	Ayesha	19	Islamabad
	1	Ali	20	Faisalabad
	3	Ahmed	21	Karachi
	2	Sara	22	Lahore
*	NULL	NULL	NULL	NULL

Descending Order

```
SELECT * FROM students
ORDER BY age DESC;
```

	id	name	age	city
▶	2	Sara	22	Lahore
	3	Ahmed	21	Karachi
	1	Ali	20	Faisalabad
	4	Ayesha	19	Islamabad
*	NULL	NULL	NULL	NULL

Limiting Results (LIMIT)

```
SELECT * FROM students
LIMIT 2;
```

	id	name	age	city
▶	1	Ali	20	Faisalabad
	2	Sara	22	Lahore
*	NULL	NULL	NULL	NULL

Returns only the first 2 rows.

Combining Conditions

```
SELECT * FROM students
WHERE age > 20 AND city = 'Lahore';
```

	id	name	age	city
▶	2	Sara	22	Lahore
✱	NULL	NULL	NULL	NULL

Full Example

```
CREATE DATABASE school;
```

```
USE school;
```

```
CREATE TABLE students (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    name VARCHAR(100),  
    age INT,  
    city VARCHAR(50)  
);
```

```
INSERT INTO students (name, age, city)  
VALUES  
('Ali', 20, 'Faisalabad'),  
('Sara', 22, 'Lahore');
```

```
SELECT * FROM students;
```

Example Output

	id	name	age	city
▶	1	Ali	20	Faisalabad
	2	Sara	22	Lahore
✱	NULL	NULL	NULL	NULL

LAB 4: DATA CONSTRAINTS IN SQL

Data Constraints in SQL

Constraints are rules applied to table columns to control what data can be stored. They help maintain **data accuracy, consistency, and integrity**.

In databases like MySQL and PostgreSQL, constraints are commonly used when creating tables.

Why Constraints Are Important

Constraints help:

- Prevent invalid data
- Avoid duplicate records
- Maintain relationships between tables
- Ensure required fields are filled

Common SQL Constraints

Constraint	Purpose
NOT NULL	Prevents empty values
UNIQUE	Prevents duplicate values
PRIMARY KEY	Unique identifier for each row
FOREIGN KEY	Links tables together
DEFAULT	Sets a default value
CHECK	Restricts allowed values
AUTO_INCREMENT	Automatically generates numbers

1. NOT NULL Constraint

Ensures a column cannot contain NULL.

```
CREATE TABLE students (  
    id INT,  
    name VARCHAR(100) NOT NULL
```

```
);
```

Trying to insert an empty name will cause an error.

2. UNIQUE Constraint

Ensures all values in a column are different.

```
CREATE TABLE users (  
    id INT,  
    email VARCHAR(100) UNIQUE  
);
```

Duplicate emails are not allowed.

3. PRIMARY KEY Constraint

A combination of:

- NOT NULL
- UNIQUE

Each table should usually have one primary key.

```
CREATE TABLE employees (  
    emp_id INT PRIMARY KEY,  
    emp_name VARCHAR(100)  
);
```

AUTO_INCREMENT with PRIMARY KEY

In MySQL:

```
CREATE TABLE employees (  
    emp_id INT AUTO_INCREMENT PRIMARY KEY,
```

```
    emp_name VARCHAR(100)
);
```

IDs increase automatically.

4. DEFAULT Constraint

Provides a default value if none is supplied.

```
CREATE TABLE products (
    product_id INT PRIMARY KEY,
    stock INT DEFAULT 0
);
```

If stock isn't entered, it becomes 0.

5. CHECK Constraint

Restricts allowed values.

```
CREATE TABLE students (
    id INT PRIMARY KEY,
    age INT CHECK (age >= 18)
);
```

Only ages 18 or higher are allowed.

6. FOREIGN KEY Constraint

Creates a relationship between two tables.

Parent Table

```
CREATE TABLE departments (
    dept_id INT PRIMARY KEY,
    dept_name VARCHAR(100)
```

```
);
```

Child Table

```
CREATE TABLE employees (  
    emp_id INT PRIMARY KEY,  
    emp_name VARCHAR(100),  
    dept_id INT,  
    FOREIGN KEY (dept_id)  
    REFERENCES departments(dept_id)  
);
```

dept_id in employees must exist in departments.

Combining Constraints

```
CREATE TABLE users (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    username VARCHAR(50) NOT NULL UNIQUE,  
    age INT CHECK (age >= 18),  
    status VARCHAR(20) DEFAULT 'active'  
);
```

Adding Constraints Later

Add UNIQUE constraint

```
ALTER TABLE users  
ADD UNIQUE (email);
```

Add FOREIGN KEY

```
ALTER TABLE employees  
ADD FOREIGN KEY (dept_id)
```



```
REFERENCES departments(dept_id);
```

Removing Constraints

Drop UNIQUE constraint (MySQL)

```
ALTER TABLE users
```

```
DROP INDEX email;
```

Summary Table

Constraint	Example
NOT NULL	name VARCHAR(50) NOT NULL
UNIQUE	email VARCHAR(100) UNIQUE
PRIMARY KEY	id INT PRIMARY KEY
DEFAULT	status DEFAULT 'active'
CHECK	CHECK(age >= 18)
FOREIGN KEY	REFERENCES departments(id)

Best Practices

- Always use a PRIMARY KEY
- Use NOT NULL for required fields
- Use FOREIGN KEY to maintain relationships
- Use CHECK to validate input
- Use UNIQUE for emails/usernames

Example:

```
CREATE DATABASE company_db;
```

```
USE company_db;
```

```
CREATE TABLE departments (  
    dept_id INT AUTO_INCREMENT PRIMARY KEY,  
    dept_name VARCHAR(100) NOT NULL UNIQUE
```

```
);  
  
CREATE TABLE employees (  
    emp_id INT AUTO_INCREMENT PRIMARY KEY,  
    emp_name VARCHAR(100) NOT NULL  
    email VARCHAR(100) UNIQUE  
    age INT CHECK (age >= 18)  
    salary FLOAT DEFAULT 30000,  
    dept_id INT,  
    FOREIGN KEY (dept_id)  
    REFERENCES departments(dept_id)  
);  
  
-- Insert data into departments table  
INSERT INTO departments (dept_name)  
VALUES  
('HR'),  
('IT'),  
('Finance');  
  
-- Insert data into employees table  
INSERT INTO employees (emp_name, email, age, salary, dept_id)  
VALUES  
('Ali', 'ali@gmail.com', 25, 50000, 1),  
('Sara', 'sara@gmail.com', 30, 70000, 2),  
('Ahmed', 'ahmed@gmail.com', 22, DEFAULT, 3);
```

Output:

	dept_id	dept_name
▶	3	Finance
	1	HR
	2	IT
*	NULL	NULL

	emp_id	emp_name	email	age	salary	dept_id
▶	1	Ali	ali@gmail.com	25	50000	1
	2	Sara	sara@gmail.com	30	70000	2
	3	Ahmed	ahmed@gmail.com	22	30000	3
*	NULL	NULL	NULL	NULL	NULL	NULL

LAB 5: DATA RETRIEVAL USING 'WHERE'

Data Retrieval Using WHERE in SQL

The WHERE clause is used to filter records based on conditions in MySQL.

Syntax

```
SELECT column_name
```

```
FROM table_name
```

```
WHERE condition;
```

Examples Using employees Table

Retrieve All Employees

```
SELECT * FROM employees;
```

	emp_id	emp_name	email	age	salary	dept_id
▶	1	Ali	ali@gmail.com	25	50000	1
	2	Sara	sara@gmail.com	30	70000	2
	3	Ahmed	ahmed@gmail.com	22	30000	3
	4	Ayesha	ayesha@gmail.com	28	45000	1
	5	Usman	usman@gmail.com	35	80000	2

employees 4 ×

1. Retrieve Employees with Salary Greater Than 40000

```
SELECT * FROM employees
```

```
WHERE salary > 40000;
```

	emp_id	emp_name	email	age	salary	dept_id
▶	1	Ali	ali@gmail.com	25	50000	1
	2	Sara	sara@gmail.com	30	70000	2
	4	Ayesha	ayesha@gmail.com	28	45000	1
	5	Usman	usman@gmail.com	35	80000	2
*	NULL	NULL	NULL	NULL	NULL	NULL

2. Retrieve Employee Named 'Ali'

```
SELECT * FROM employees  
WHERE emp_name = 'Ali';
```

	emp_id	emp_name	email	age	salary	dept_id
▶	1	Ali	ali@gmail.com	25	50000	1
*	NULL	NULL	NULL	NULL	NULL	NULL

3. Retrieve Employees Older Than 24

```
SELECT * FROM employees  
WHERE age > 24;
```

	emp_id	emp_name	email	age	salary	dept_id
▶	2	Sara	sara@gmail.com	30	70000	2
	4	Ayesha	ayesha@gmail.com	28	45000	1
	5	Usman	usman@gmail.com	35	80000	2
*	NULL	NULL	NULL	NULL	NULL	NULL

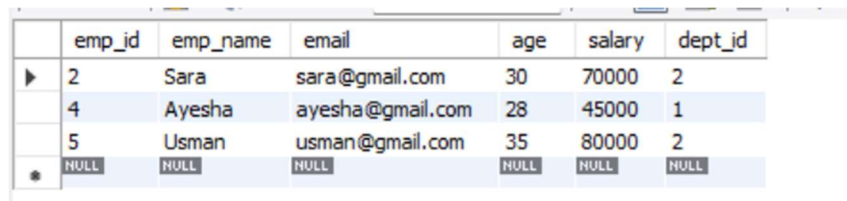
4. Retrieve Employees from Department 2

```
SELECT * FROM employees  
WHERE dept_id = 2;
```

	emp_id	emp_name	email	age	salary	dept_id
▶	2	Sara	sara@gmail.com	30	70000	2
	5	Usman	usman@gmail.com	35	80000	2
*	NULL	NULL	NULL	NULL	35	NULL

5. Using AND Operator

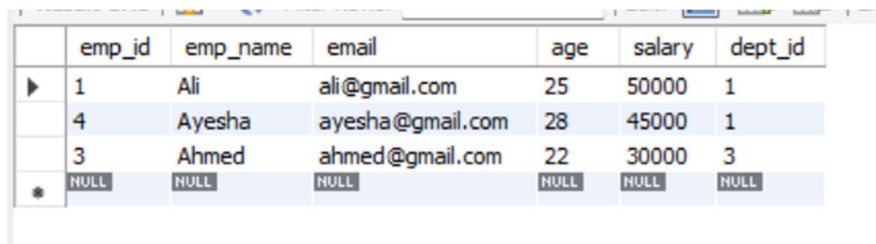
```
SELECT * FROM employees  
WHERE salary > 40000  
AND age > 25;
```



	emp_id	emp_name	email	age	salary	dept_id
▶	2	Sara	sara@gmail.com	30	70000	2
	4	Ayesha	ayesha@gmail.com	28	45000	1
	5	Usman	usman@gmail.com	35	80000	2
*	NULL	NULL	NULL	NULL	NULL	NULL

6. Using OR Operator

```
SELECT * FROM employees  
WHERE dept_id = 1  
OR dept_id = 2;
```



	emp_id	emp_name	email	age	salary	dept_id
▶	1	Ali	ali@gmail.com	25	50000	1
	4	Ayesha	ayesha@gmail.com	28	45000	1
	3	Ahmed	ahmed@gmail.com	22	30000	3
*	NULL	NULL	NULL	NULL	NULL	NULL

7. Using NOT Operator

```
SELECT * FROM employees  
WHERE NOT dept_id = 3;
```

	emp_id	emp_name	email	age	salary	dept_id
▶	1	Ali	ali@gmail.com	25	50000	1
	4	Ayesha	ayesha@gmail.com	28	45000	1
	3	Ahmed	ahmed@gmail.com	22	30000	3
*	NULL	NULL	NULL	NULL	NULL	NULL

8. Using LIKE (Search Pattern)

Names starting with 'A'

```
SELECT * FROM employees
WHERE emp_name LIKE 'A%';
```

	emp_id	emp_name	email	age	salary	dept_id
▶	1	Ali	ali@gmail.com	25	50000	1
	3	Ahmed	ahmed@gmail.com	22	30000	3
	4	Ayesha	ayesha@gmail.com	28	45000	1
*	NULL	NULL	NULL	NULL	NULL	NULL

9. Using BETWEEN

```
SELECT * FROM employees
WHERE salary BETWEEN 30000 AND 60000;
```

	emp_id	emp_name	email	age	salary	dept_id
▶	1	Ali	ali@gmail.com	25	50000	1
	2	Sara	sara@gmail.com	30	70000	2
	3	Ahmed	ahmed@gmail.com	22	30000	3
	4	Ayesha	ayesha@gmail.com	28	45000	1
*	NULL	NULL	NULL	NULL	NULL	NULL

10. Using IN

```
SELECT * FROM employees
WHERE dept_id IN (1, 3);
```

	emp_id	emp_name	email	age	salary	dept_id
▶	1	Ali	ali@gmail.com	25	50000	1
	4	Ayesha	ayesha@gmail.com	28	45000	1
	2	Sara	sara@gmail.com	30	70000	2
	5	Usman	usman@gmail.com	35	80000	2
*	NULL	NULL	NULL	NULL	NULL	NULL

11. Using IS NULL

```
SELECT * FROM employees
WHERE email IS NULL;
```

	emp_id	emp_name	email	age	salary	dept_id
*	NULL	NULL	NULL	NULL	NULL	NULL

Combined Example

```
SELECT emp_name, salary
FROM employees
WHERE salary > 30000
AND age >= 25;
```

	emp_name	salary
▶	Ali	50000
	Sara	70000
	Ayesha	45000
	Usman	80000

Common Comparison Operators

Operator	Meaning
=	Equal
!= or <>	Not equal

>	Greater than
<	Less than
>=	Greater or equal
<=	Less or equal

LAB 6: SORTING AND FILTERING DATA IN SQL

Examples Using employees Table

Retrieve All Employees

```
SELECT * FROM employees;
```

	emp_id	emp_name	email	age	salary	dept_id
▶	1	Ali	ali@gmail.com	25	50000	1
	2	Sara	sara@gmail.com	30	70000	2
	3	Ahmed	ahmed@gmail.com	22	30000	3
	4	Ayesha	ayesha@gmail.com	28	45000	1
	5	Usman	usman@gmail.com	35	80000	2

employees 4 ×

1. Retrieve Employees with Salary Greater Than 40000

```
SELECT * FROM employees
```

```
WHERE salary > 40000;
```

	emp_id	emp_name	email	age	salary	dept_id
▶	1	Ali	ali@gmail.com	25	50000	1
	2	Sara	sara@gmail.com	30	70000	2
	4	Ayesha	ayesha@gmail.com	28	45000	1
	5	Usman	usman@gmail.com	35	80000	2
*	NULL	NULL	NULL	NULL	NULL	NULL

2. Retrieve Employee Named 'Ali'

```
SELECT * FROM employees
```

```
WHERE emp_name = 'Ali';
```

	emp_id	emp_name	email	age	salary	dept_id
▶	1	Ali	ali@gmail.com	25	50000	1
*	NULL	NULL	NULL	NULL	NULL	NULL

3. Retrieve Employees Older Than 24

```
SELECT * FROM employees
```

```
WHERE age > 24;
```

	emp_id	emp_name	email	age	salary	dept_id
▶	2	Sara	sara@gmail.com	30	70000	2
	4	Ayesha	ayesha@gmail.com	28	45000	1
	5	Usman	usman@gmail.com	35	80000	2
*	NULL	NULL	NULL	NULL	NULL	NULL

4. Retrieve Employees from Department 2

```
SELECT * FROM employees
```

```
WHERE dept_id = 2;
```

	emp_id	emp_name	email	age	salary	dept_id
▶	2	Sara	sara@gmail.com	30	70000	2
	5	Usman	usman@gmail.com	35	80000	2
*	NULL	NULL	NULL	NULL	35	NULL

5. Using AND Operator

```
SELECT * FROM employees
```

```
WHERE salary > 40000
```

```
AND age > 25;
```

	emp_id	emp_name	email	age	salary	dept_id
▶	2	Sara	sara@gmail.com	30	70000	2
	4	Ayesha	ayesha@gmail.com	28	45000	1
	5	Usman	usman@gmail.com	35	80000	2
*	NULL	NULL	NULL	NULL	NULL	NULL

6. Using OR Operator

SELECT * FROM employees

WHERE dept_id = 1

OR dept_id = 2;

	emp_id	emp_name	email	age	salary	dept_id
▶	1	Ali	ali@gmail.com	25	50000	1
	4	Ayesha	ayesha@gmail.com	28	45000	1
	3	Ahmed	ahmed@gmail.com	22	30000	3
*	NULL	NULL	NULL	NULL	NULL	NULL

7. Using NOT Operator

SELECT * FROM employees

WHERE NOT dept_id = 3;

	emp_id	emp_name	email	age	salary	dept_id
▶	1	Ali	ali@gmail.com	25	50000	1
	4	Ayesha	ayesha@gmail.com	28	45000	1
	3	Ahmed	ahmed@gmail.com	22	30000	3
*	NULL	NULL	NULL	NULL	NULL	NULL

8. Using LIKE (Search Pattern)

Names starting with 'A'

```
SELECT * FROM employees
WHERE emp_name LIKE 'A%';
```

	emp_id	emp_name	email	age	salary	dept_id
▶	1	Ali	ali@gmail.com	25	50000	1
	3	Ahmed	ahmed@gmail.com	22	30000	3
	4	Ayesha	ayesha@gmail.com	28	45000	1
*	NULL	NULL	NULL	NULL	NULL	NULL

9. Using BETWEEN

```
SELECT * FROM employees
WHERE salary BETWEEN 30000 AND 60000;
```

	emp_id	emp_name	email	age	salary	dept_id
▶	1	Ali	ali@gmail.com	25	50000	1
	2	Sara	sara@gmail.com	30	70000	2
	3	Ahmed	ahmed@gmail.com	22	30000	3
	4	Ayesha	ayesha@gmail.com	28	45000	1
*	NULL	NULL	NULL	NULL	NULL	NULL

10. Using IN

```
SELECT * FROM employees
WHERE dept_id IN (1, 3);
```

	emp_id	emp_name	email	age	salary	dept_id
▶	1	Ali	ali@gmail.com	25	50000	1
	4	Ayesha	ayesha@gmail.com	28	45000	1
	2	Sara	sara@gmail.com	30	70000	2
	5	Usman	usman@gmail.com	35	80000	2
*	NULL	NULL	NULL	NULL	NULL	NULL

11. Using IS NULL

```
SELECT * FROM employees
WHERE email IS NULL;
```

	emp_id	emp_name	email	age	salary	dept_id
*	NULL	NULL	NULL	NULL	NULL	NULL

Combined Example

```
SELECT emp_name, salary
FROM employees
WHERE salary > 40000
ORDER BY salary DESC;
```

	emp_name	salary
▶	Usman	80000
	Sara	70000
	Ali	50000
	Ayesha	45000

Operator	Meaning
=	Equal
!=	Not equal
>	Greater than
<	Less than
>=	Greater or equal
<=	Less or equal
BETWEEN	Range
LIKE	Pattern search
IN	Match multiple values

LAB 7: MODIFYING DATA AND AGGREGATE FUNCTIONS

Modifying Data and Aggregate Functions in SQL

In MySQL, you use **UPDATE**, **DELETE** to modify data and **aggregate functions** to summarize data.

1. Modifying Data

A. UPDATE (Change Existing Data)

Syntax

```
UPDATE table_name  
  
SET column1 = value1, column2 = value2  
  
WHERE condition;
```

Example: Update Salary

```
UPDATE employees  
  
SET salary = 60000  
  
WHERE emp_name = 'Ali';
```

	emp_id	emp_name	email	age	salary	dept_id
▶	1	Ali	ali@gmail.com	25	60000	1
	3	Ahmed	ahmed@gmail.com	22	30000	3
	4	Ayesha	ayesha@gmail.com	28	45000	1
*	NULL	NULL	NULL	NULL	NULL	NULL

B. DELETE (Remove Data)

Syntax

```
DELETE FROM table_name  
  
WHERE condition;
```

Example: Delete a Record

```
DELETE FROM employees
WHERE emp_name = 'Sara';
```

	emp_id	emp_name	email	age	salary	dept_id
▶	1	Ali	ali@gmail.com	25	60000	1
	3	Ahmed	ahmed@gmail.com	22	30000	3
	4	Ayesha	ayesha@gmail.com	28	45000	1
	5	Usman	usman@gmail.com	35	80000	2
*	NULL	NULL	NULL	NULL	NULL	NULL

Delete All Rows (Be Careful)

```
DELETE FROM employees;
```

2. Aggregate Functions

Aggregate functions perform calculations on multiple rows and return a single result.

A. COUNT()

Counts rows.

```
SELECT COUNT(*) AS total_employees
FROM employees;
```

	total_employees
▶	4

B. SUM()

Adds values.

```
SELECT SUM(salary) AS total_salary
FROM employees;
```


	total_salary
▶	215000

C. AVG()

Finds average value.

```
SELECT AVG(salary) AS average_salary  
FROM employees;
```

	average_salary
▶	53750

D. MAX()

Finds highest value.

```
SELECT MAX(salary) AS highest_salary  
FROM employees;
```

	highest_salary
▶	80000

E. MIN()

Finds lowest value.

```
SELECT MIN(salary) AS lowest_salary  
FROM employees;
```

	lowest_salary
▶	30000

3. Aggregate Functions with GROUP BY

Example: Salary per Department

```
SELECT dept_id, SUM(salary) AS total_salary  
FROM employees  
GROUP BY dept_id;
```

	dept_id	total_salary
▶	1	105000
	2	80000
	3	30000

Example: Count Employees per Department

```
SELECT dept_id, COUNT(*) AS total_employees  
FROM employees  
GROUP BY dept_id;
```

	dept_id	total_employees
▶	1	2
	2	1
	3	1

4. HAVING Clause (Filter Groups)

Used with aggregates (not individual rows).

```
SELECT dept_id, AVG(salary) AS avg_salary  
FROM employees  
GROUP BY dept_id  
HAVING AVG(salary) > 50000;
```

	dept_id	avg_salary
▶	1	52500
	2	80000

Summary

Modifying Data

- UPDATE → change data
- DELETE → remove data

Aggregate Functions

- COUNT() → number of rows
- SUM() → total
- AVG() → average
- MAX() → highest value
- MIN() → lowest value

LAB 8: KEYS IN DBMS

Keys in DBMS (Database Management System)

In a database, **keys** are used to uniquely identify records and establish relationships between tables. They are essential for maintaining data integrity in systems like MySQL.

1. Super Key

A **super key** is any set of columns that can uniquely identify a row.

Super keys:

- emp_id
- email
- {emp_id, name}
- {email, name}

Any combination that uniquely identifies a record.

2. Candidate Key

A **candidate key** is a *minimal super key* (no extra columns).

Example:

- emp_id
- email

Both can uniquely identify employees.

3. Primary Key

A **primary key** is the chosen candidate key used to uniquely identify rows.

Rules:

- Must be unique
- Cannot be NULL

- Only one per table

```
CREATE TABLE employees (  
    emp_id INT PRIMARY KEY,  
    email VARCHAR(100)  
);
```

4. Alternate Key

Candidate keys not chosen as primary key.

Example:

- Primary Key = emp_id
- Alternate Key = email

5. Foreign Key

A **foreign key** creates a link between two tables.

```
CREATE TABLE departments (  
    dept_id INT PRIMARY KEY,  
    dept_name VARCHAR(100)  
);
```

```
CREATE TABLE employees (  
    emp_id INT PRIMARY KEY,  
    dept_id INT,  
    FOREIGN KEY (dept_id) REFERENCES departments(dept_id)  
);
```

Ensures valid relationships between tables.

6. Composite Key

A key made of **two or more columns**.

Example:

```
CREATE TABLE enrollment (
    student_id INT,
    course_id INT,
    PRIMARY KEY (student_id, course_id)
);
```

Combination uniquely identifies each record.

7. Unique Key

Ensures all values are different (like primary key but allows NULL in some systems).

```
CREATE TABLE users (
    id INT PRIMARY KEY,
    email VARCHAR(100) UNIQUE
);
```

8. Surrogate Key

A system-generated key with no real meaning.

Example:

- Auto increment ID

```
emp_id INT AUTO_INCREMENT PRIMARY KEY
```

Summary Table

Key Type	Purpose
----------	---------

Super Key	Any unique identifier set
Candidate Key	Minimal unique key
Primary Key	Main unique identifier
Alternate Key	Other candidate keys
Foreign Key	Links tables
Composite Key	Multiple columns as key
Unique Key	Ensures uniqueness
Surrogate Key	System-generated ID

Quick Example (All Keys Together)

```
CREATE TABLE departments (
    dept_id INT PRIMARY KEY,
    dept_name VARCHAR(100) UNIQUE
);
```

```
CREATE TABLE employees (
    emp_id INT AUTO_INCREMENT PRIMARY KEY,
    email VARCHAR(100) UNIQUE,
    name VARCHAR(100) NOT NULL,
    dept_id INT,

    FOREIGN KEY (dept_id) REFERENCES departments(dept_id)
);
```

Select:

```
SELECT
    employees.name,
    employees.email,
    departments.dept_name
FROM employees
```

JOIN departments

ON employees.dept_id = departments.dept_id;

	name	email	dept_name
▶	Ahmed	ahmed@gmail.com	Finance
	Sara	sara@gmail.com	HR
	Ali	ali@gmail.com	IT
	Ayesha	ayesha@gmail.com	IT
	Usman	usman@gmail.com	Marketing

Result 29 ×

LAB 9: FILTERING DATA USING 'WHERE' AND 'LIKE'

1. Filtering with WHERE

WHERE is used to filter rows based on conditions.

Basic Syntax

```
SELECT column_name
```

```
FROM table_name
```

```
WHERE condition;
```

Examples (Using employees table)

1. Employees in IT department

```
SELECT * FROM employees
```

```
WHERE dept_id = 2;
```

	emp_id	email	name	dept_id
▶	1	ali@gmail.com	Ali	2
	4	ayesha@gmail.com	Ayesha	2
*	NULL	NULL	NULL	NULL

2. Employees with salary greater than 40000

```
SELECT * FROM employees
```

```
WHERE salary > 40000;
```

	emp_id	email	name	dept_id	salary
▶	1	ali@gmail.com	Ali	2	50000
	2	sara@gmail.com	Sara	1	45000
	3	ahmed@gmail.com	Ahmed	3	60000
	4	ayesha@gmail.com	Ayesha	2	55000
	5	usman@gmail.com	Usman	4	70000
*	NULL	NULL	NULL	NULL	NULL

3. Employee named Ali

SELECT * FROM employees

WHERE name = 'Ali';

	emp_id	email	name	dept_id	salary
▶	1	ali@gmail.com	Ali	2	50000
*	NULL	NULL	NULL	NULL	NULL

4. Multiple conditions (AND)

SELECT * FROM employees

WHERE dept_id = 2 AND salary > 30000;

	emp_id	email	name	dept_id	salary
▶	1	ali@gmail.com	Ali	2	50000
	4	ayesha@gmail.com	Ayesha	2	55000
*	NULL	NULL	NULL	NULL	NULL

5. Multiple conditions (OR)

SELECT * FROM employees

WHERE dept_id = 1 OR dept_id = 3;

	emp_id	email	name	dept_id	salary
▶	2	sara@gmail.com	Sara	1	45000
	3	ahmed@gmail.com	Ahmed	3	60000
*	NULL	NULL	NULL	NULL	NULL

2. Filtering with LIKE

LIKE is used for **pattern matching in text**.

Wildcards

Symbol	Meaning
%	Any number of characters
_	Single character

Examples

1. Names starting with 'A'

```
SELECT * FROM employees
WHERE name LIKE 'A%';
```

	emp_id	email	name	dept_id	salary
▶	1	ali@gmail.com	Ali	2	50000
	3	ahmed@gmail.com	Ahmed	3	60000
	4	ayesha@gmail.com	Ayesha	2	55000
*	NULL	NULL	NULL	NULL	NULL

2. Names ending with 'a'

```
SELECT * FROM employees
WHERE name LIKE '%a';
```

	emp_id	email	name	dept_id	salary
▶	2	sara@gmail.com	Sara	1	45000
	4	ayesha@gmail.com	Ayesha	2	55000
*	NULL	NULL	NULL	NULL	NULL

3. Names containing 'a'

SELECT * FROM employees

WHERE name LIKE '%a%';

	emp_id	email	name	dept_id	salary
▶	1	ali@gmail.com	Ali	2	50000
	2	sara@gmail.com	Sara	1	45000
	3	ahmed@gmail.com	Ahmed	3	60000
	4	ayesha@gmail.com	Ayesha	2	55000
	5	usman@gmail.com	Usman	4	70000

4. Names with exactly 4 letters

SELECT * FROM employees

WHERE name LIKE '____';

	emp_id	email	name	dept_id	salary
▶	2	sara@gmail.com	Sara	1	45000
*	NULL	NULL	NULL	NULL	NULL

5. Names starting with 'S' and 3 more letters

SELECT * FROM employees

WHERE name LIKE 'S____';

	emp_id	email	name	dept_id	salary
▶	2	sara@gmail.com	Sara	1	45000
*	NULL	NULL	NULL	NULL	NULL

6. Emails from Gmail

```
SELECT * FROM employees
```

```
WHERE email LIKE '%@gmail.com';
```

	emp_id	email	name	dept_id	salary
▶	1	ali@gmail.com	Ali	2	50000
	2	sara@gmail.com	Sara	1	45000
	3	ahmed@gmail.com	Ahmed	3	60000
	4	ayesha@gmail.com	Ayesha	2	55000
	5	usman@gmail.com	Usman	4	70000
*	NULL	NULL	NULL	NULL	NULL

7. Exclude names starting with A

```
SELECT * FROM employees
```

```
WHERE name NOT LIKE 'A%';
```

	emp_id	email	name	dept_id	salary
▶	2	sara@gmail.com	Sara	1	45000
	5	usman@gmail.com	Usman	4	70000
*	NULL	NULL	NULL	NULL	NULL

3. Combining WHERE + LIKE

Employees in IT whose name starts with A

```
SELECT * FROM employees
```

```
WHERE dept_id = 2
```

```
AND name LIKE 'A%';
```

	emp_id	email	name	dept_id	salary
▶	1	ali@gmail.com	Ali	2	50000
	4	ayesha@gmail.com	Ayesha	2	55000
★	NULL	NULL	NULL	NULL	NULL

Employees earning more than 30000 and name contains 'a'

```
SELECT * FROM employees
```

```
WHERE salary > 30000
```

```
AND name LIKE '%a%';
```

	emp_id	email	name	dept_id	salary
▶	1	ali@gmail.com	Ali	2	50000
	2	sara@gmail.com	Sara	1	45000
	3	ahmed@gmail.com	Ahmed	3	60000
	4	ayesha@gmail.com	Ayesha	2	55000
	5	usman@gmail.com	Usman	4	70000
★	NULL	NULL	NULL	NULL	NULL

Summary

WHERE

- Filters rows using conditions
- Works with numbers, text, and dates

LIKE

- Used for pattern matching
- % = many characters
- _ = single character

LAB 10: ADVANCED DATA FILTERING I

1. ORDER BY (Sorting Data)

Used to sort results in ascending or descending order.

Syntax

```
SELECT * FROM employees  
ORDER BY column_name ASC|DESC;
```

Examples

Sort by salary (low to high)

```
SELECT * FROM employees  
ORDER BY salary ASC;
```

	emp_id	email	name	dept_id	salary
▶	2	sara@gmail.com	Sara	1	45000
	1	ali@gmail.com	Ali	2	50000
	4	ayesha@gmail.com	Ayesha	2	55000
	3	ahmed@gmail.com	Ahmed	3	60000
	5	usman@gmail.com	Usman	4	70000
*	NULL	NULL	NULL	NULL	NULL

Sort by salary (high to low)

```
SELECT * FROM employees  
ORDER BY salary DESC;
```

	emp_id	email	name	dept_id	salary
▶	5	usman@gmail.com	Usman	4	70000
	3	ahmed@gmail.com	Ahmed	3	60000
	4	ayesha@gmail.com	Ayesha	2	55000
	1	ali@gmail.com	Ali	2	50000
	2	sara@gmail.com	Sara	1	45000
*	NULL	NULL	NULL	NULL	NULL

Sort by name alphabetically

```
SELECT * FROM employees
```

```
ORDER BY name ASC;
```

	emp_id	email	name	dept_id	salary
▶	3	ahmed@gmail.com	Ahmed	3	60000
	1	ali@gmail.com	Ali	2	50000
	4	ayesha@gmail.com	Ayesha	2	55000
	2	sara@gmail.com	Sara	1	45000
	5	usman@gmail.com	Usman	4	70000
*	NULL	NULL	NULL	NULL	NULL

2. IN (Multiple Values Filter)

Used instead of many OR conditions.

Syntax

```
SELECT * FROM employees
```

```
WHERE column_name IN (value1, value2, value3);
```

Examples

Employees in dept 1, 2, or 3

```
SELECT * FROM employees
```

```
WHERE dept_id IN (1, 2, 3);
```


	emp_id	email	name	dept_id	salary
▶	2	sara@gmail.com	Sara	1	45000
	1	ali@gmail.com	Ali	2	50000
	4	ayesha@gmail.com	Ayesha	2	55000
	3	ahmed@gmail.com	Ahmed	3	60000
★	NULL	NULL	NULL	NULL	NULL

Employees named Ali, Sara, or Ahmed

```
SELECT * FROM employees
```

```
WHERE name IN ('Ali', 'Sara', 'Ahmed');
```

	emp_id	email	name	dept_id	salary
▶	1	ali@gmail.com	Ali	2	50000
	2	sara@gmail.com	Sara	1	45000
	3	ahmed@gmail.com	Ahmed	3	60000
★	NULL	NULL	NULL	NULL	NULL

NOT IN (exclude departments)

```
SELECT * FROM employees
```

```
WHERE dept_id NOT IN (2);
```

	emp_id	email	name	dept_id	salary
▶	2	sara@gmail.com	Sara	1	45000
	3	ahmed@gmail.com	Ahmed	3	60000
	5	usman@gmail.com	Usman	4	70000
★	NULL	NULL	NULL	NULL	NULL

3. BETWEEN (Range Filtering)

Used for numeric or date ranges.

Syntax

```
SELECT * FROM employees
WHERE column_name BETWEEN value1 AND value2;
```

Examples

Employees with salary between 40000 and 60000

```
SELECT * FROM employees
WHERE salary BETWEEN 40000 AND 60000;
```

	emp_id	email	name	dept_id	salary
▶	1	ali@gmail.com	Ali	2	50000
	2	sara@gmail.com	Sara	1	45000
	3	ahmed@gmail.com	Ahmed	3	60000
	4	ayesha@gmail.com	Ayesha	2	55000
*	NULL	NULL	NULL	NULL	NULL

NOT BETWEEN (outside range)

```
SELECT * FROM employees
WHERE salary NOT BETWEEN 40000 AND 60000;
```

	emp_id	email	name	dept_id	salary
▶	5	usman@gmail.com	Usman	4	70000
*	NULL	NULL	NULL	NULL	NULL

4. Combined Example (Very Important)

Filter + IN + ORDER BY

```
SELECT * FROM employees
WHERE dept_id IN (1, 2)
ORDER BY salary DESC;
```

	emp_id	email	name	dept_id	salary
▶	4	ayesha@gmail.com	Ayesha	2	55000
	1	ali@gmail.com	Ali	2	50000
	2	sara@gmail.com	Sara	1	45000
*	NULL	NULL	NULL	NULL	NULL

BETWEEN + ORDER BY

SELECT * FROM employees

WHERE salary BETWEEN 30000 AND 70000

ORDER BY salary ASC;

	emp_id	email	name	dept_id	salary
▶	2	sara@gmail.com	Sara	1	45000
	1	ali@gmail.com	Ali	2	50000
	4	ayesha@gmail.com	Ayesha	2	55000
	3	ahmed@gmail.com	Ahmed	3	60000
	5	usman@gmail.com	Usman	4	70000
*	NULL	NULL	NULL	NULL	NULL

employees 53 ×

Quick Summary

Clause	Purpose
ORDER BY	Sort data
IN	Match multiple values
BETWEEN	Range filtering

LAB 11: ADVANCED DATA FILTERING II

1. Aggregate Functions

Aggregate functions perform calculations on multiple rows and return a single result.

A. COUNT()

Counts rows.

```
SELECT COUNT(*) AS total_employees  
FROM employees;
```

	total_employees
▶	4

B. SUM()

Adds values.

```
SELECT SUM(salary) AS total_salary  
FROM employees;
```

	total_salary
▶	215000

C. AVG()

Finds average value.

```
SELECT AVG(salary) AS average_salary  
FROM employees;
```

	average_salary
▶	53750

D. MAX()

Finds highest value.

```
SELECT MAX(salary) AS highest_salary  
FROM employees;
```

	highest_salary
▶	80000

E. MIN()

Finds lowest value.

```
SELECT MIN(salary) AS lowest_salary  
FROM employees;
```

	lowest_salary
▶	30000

2. Aggregate Functions with GROUP BY

Example: Salary per Department

```
SELECT dept_id, SUM(salary) AS total_salary  
FROM employees  
GROUP BY dept_id;
```

	dept_id	total_salary
▶	1	105000
	2	80000
	3	30000

Example: Count Employees per Department

```
SELECT dept_id, COUNT(*) AS total_employees  
FROM employees  
GROUP BY dept_id;
```

	dept_id	total_employees
▶	1	2
	2	1
	3	1

3. HAVING Clause (Filter Groups)

Used with aggregates (not individual rows).

```
SELECT dept_id, AVG(salary) AS avg_salary  
FROM employees  
GROUP BY dept_id  
HAVING AVG(salary) > 50000;
```

	dept_id	avg_salary
▶	1	52500
	2	80000

Summary

Aggregate Functions

- COUNT() → number of rows
- SUM() → total

- `AVG()` → average
- `MAX()` → highest value
- `MIN()` → lowest value

LAB 12: NORMALIZATION

Normalization in DBMS

Normalization is the process of organizing data in a database to reduce **redundancy (duplicate data)** and improve **data integrity**.

It is used in relational databases like MySQL.

Why Normalization is Needed

Without normalization:

- Data is repeated (waste of space)
- Updates become difficult
- Insertion and deletion anomalies occur

Example of Bad (Unnormalized) Table

student_id	name	course	teacher
1	Ali	SQL	Mr. Khan
2	Ali	Python	Mr. Ahmed
3	Sara	SQL	Mr. Khan

Problems:

- "Ali" repeats
- "SQL → Mr. Khan" repeats
- If teacher changes, multiple rows must be updated

1NF (First Normal Form)

Rule:

- Each column must have **atomic (single) values**
- No repeating groups

Already in 1NF:

student_id	name	course	teacher
------------	------	--------	---------

No multiple values in one cell

2NF (Second Normal Form)

Rule:

- Must be in 1NF
- Remove **partial dependency**

Problem:

Teacher depends on course, not student.

Split into tables:

Students Table

student_id	name
1	Ali
2	Sara

Courses Table

course	teacher
SQL	Mr. Khan
Python	Mr. Ahmed

Enrollment Table

student_id	course
1	SQL
1	Python
2	SQL

3NF (Third Normal Form)

Rule:

- Must be in 2NF
- Remove **transitive dependency**

Example:

If teacher depends on course, not directly on student → separate it.

Already fixed in above structure

Final Normalized Structure

Students

- student_id (PK)
- name

Courses

- course_id (PK)
- course_name
- teacher

Enrollment

- student_id (FK)
- course_id (FK)

Benefits of Normalization

- Removes duplicate data
- Saves storage
- Improves consistency
- Makes updates easier
- Reduces anomalies

Types of Anomalies Prevented

1. Insertion Anomaly

Cannot insert data without unnecessary fields

2. Update Anomaly

Same data must be updated in multiple places

3. Deletion Anomaly

Deleting one record removes useful information

Simple Real-Life Example

Instead of storing this:

“Ali → SQL → Mr. Khan” (repeated many times)

We store:

- Ali in Students table
- SQL in Courses table
- Relationship in Enrollment table

Summary

Normal Form	Goal
1NF	Atomic values

2NF	Remove partial dependency
3NF	Remove transitive dependency

LAB 13: INNER AND CROSS JOINS

1. INNER JOIN

What it does

INNER JOIN returns only the **matching rows** between two tables based on a condition.

Syntax

```
SELECT columns  
FROM table1  
INNER JOIN table2  
ON table1.column = table2.column;
```

Example (Employees + Departments)

```
SELECT  
    employees.name,  
    employees.email,  
    departments.dept_name  
FROM employees  
INNER JOIN departments  
ON employees.dept_id = departments.dept_id;
```

	name	email	dept_name
▶	Ahmed	ahmed@gmail.com	Finance
	Sara	sara@gmail.com	HR
	Ali	ali@gmail.com	IT
	Ayesha	ayesha@gmail.com	IT
	Usman	usman@gmail.com	Marketing

Key Point

- Only shows employees that have a valid department
- If no match → row is NOT shown

2. CROSS JOIN

What it does

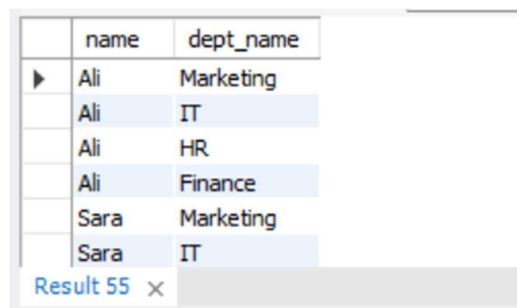
CROSS JOIN returns the **Cartesian product** (all possible combinations of rows).

Syntax

```
SELECT *
FROM table1
CROSS JOIN table2;
```

Example

```
SELECT
    employees.name,
    departments.dept_name
FROM employees
CROSS JOIN departments;
```



The screenshot shows a query result window titled 'Result 55'. It displays a table with two columns: 'name' and 'dept_name'. The data rows are as follows:

name	dept_name
Ali	Marketing
Ali	IT
Ali	HR
Ali	Finance
Sara	Marketing
Sara	IT

Key Point

- Every employee combines with every department
- No condition required

Difference Between INNER JOIN and CROSS JOIN

Feature	INNER JOIN	CROSS JOIN
Matching required	Yes	No
Output size	Smaller	Very large
Condition (ON)	Required	Not needed

Use case	Real relationships	Combinations
----------	--------------------	--------------

Quick Real-Life Meaning

INNER JOIN

“Show employees with their correct departments”

CROSS JOIN

“Show every employee with every department (all possibilities)”

Simple Summary

- **INNER JOIN** → matches only related data
- **CROSS JOIN** → all combinations of data

LAB 14: OUTER JOIN AND IT'S TYPES

OUTER JOIN and Its Types in SQL

In MySQL, **OUTER JOIN** is used to return **matching rows + non-matching rows** from one or both tables.

It is mainly used to avoid losing data when matches don't exist.

We use it with tables like:

- employees
- departments

1. LEFT OUTER JOIN (LEFT JOIN)

What it does

Returns:

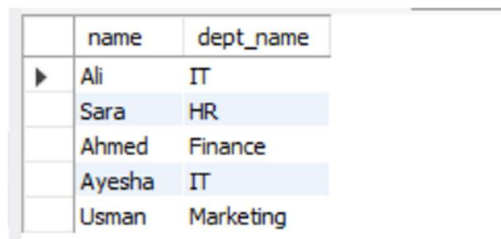
- All rows from **left table**
- Matching rows from right table
- Non-matching rows → NULL

Syntax

```
SELECT columns  
FROM table1  
LEFT JOIN table2  
ON table1.column = table2.column;
```

Example

```
SELECT  
    employees.name,  
    departments.dept_name  
FROM employees  
LEFT JOIN departments  
ON employees.dept_id = departments.dept_id;
```



	name	dept_name
▶	Ali	IT
	Sara	HR
	Ahmed	Finance
	Ayesha	IT
	Usman	Marketing

2. RIGHT OUTER JOIN (RIGHT JOIN)

What it does

Returns:

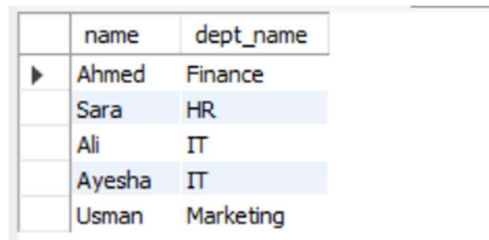
- All rows from **right table**
- Matching rows from left table
- Non-matching rows → NULL

Syntax


```
SELECT columns  
FROM table1  
RIGHT JOIN table2  
ON table1.column = table2.column;
```

Example

```
SELECT  
    employees.name,  
    departments.dept_name  
FROM employees  
RIGHT JOIN departments  
ON employees.dept_id = departments.dept_id;
```



A screenshot of a database query result. It shows a table with two columns: 'name' and 'dept_name'. The data is as follows:

	name	dept_name
▶	Ahmed	Finance
	Sara	HR
	Ali	IT
	Ayesha	IT
	Usman	Marketing

3. FULL OUTER JOIN (Concept)

What it does

Returns:

- All rows from both tables
- Matching + non-matching rows

Important Note

MySQL does **NOT** directly support **FULL OUTER JOIN**

But conceptually:

SELECT *

FROM employees

LEFT JOIN departments ON employees.dept_id = departments.dept_id

UNION

SELECT *

FROM employees

RIGHT JOIN departments ON employees.dept_id = departments.dept_id;

	emp_id	email	name	dept_id	salary	dept_id	dept_name
▶	1	ali@gmail.com	Ali	2	50000	2	IT
	2	sara@gmail.com	Sara	1	45000	1	HR
	3	ahmed@gmail.com	Ahmed	3	60000	3	Finance
	4	ayesha@gmail.com	Ayesha	2	55000	2	IT
	5	usman@gmail.com	Usman	4	70000	4	Marketing

4. Quick Comparison

Type	Includes Left Table	Includes Right Table	Unmatched Data
LEFT JOIN	✓	✗	Left side
RIGHT JOIN	✗	✓	Right side
FULL OUTER JOIN	✓	✓	Both sides

5. Easy Real-Life Meaning

- **LEFT JOIN** → “Show all employees, even if no department”

- **RIGHT JOIN** → “Show all departments, even if no employees”
- **FULL JOIN** → “Show everything from both sides”

Summary

- **OUTER JOIN** = includes unmatched rows
- **LEFT JOIN** = left table is priority
- **RIGHT JOIN** = right table is priority
- **FULL JOIN** = both tables fully combined